Goal — a savings goal that feels real

Epic: EPIC-012 (#160) · **Requirements:** A-19, A-23, A-24, A-03 · **Decisions:** ADR-0006 goal projection, schema §3.7 and §6.3

Problem

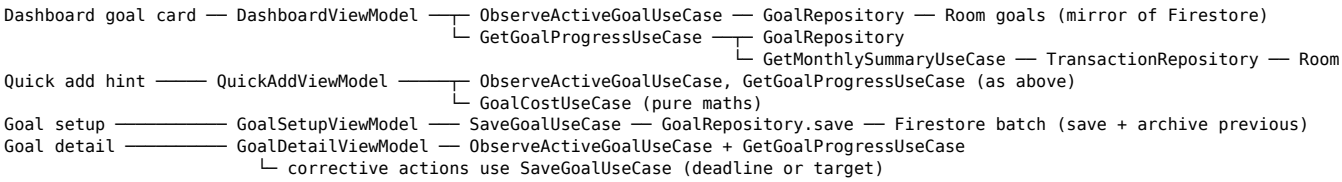
Kavindu's MacBook goal is "emotionally present and financially invisible": he does not know what he must save each month or what he actually saves (**A-19**). His income is irregular, so a deadline he set in a good month can quietly become impossible (**A-24**). He is impulsive with discretionary spending and avoidant about bad news (**A-23**), so the goal has to show the cost of a spend at the moment he makes it, in a neutral tone, never as a red failure state (**A-03**).

Design decision

- **One active goal**, stored at `users/{uid}/goals/{id}`. Saving a new active goal archives the previous one in the same batch, so there are never two (schema §3.7).
- **Progress is computed, never stored.** `GetGoalProgressUseCase` derives everything from the goal and the monthly summaries since it started (A-01):
  - `savedToDate` = the goal's baseline (money already saved when it started) + actual net saving since its start month. Progress is not tied to one account's balance, because Kavindu's savings move between pots (A-21).
  - `requiredMonthly` = `remaining ÷ months left`, rounded **up**, so the app never under-asks.
  - `actualPace` = the average net saving over the last up to 3 **complete** months. Before the first complete month the status is `NOT_ENOUGH_DATA`, not "behind".
  - **Status** is AHEAD (≥ 110 % of required), ON_TRACK (≥ 90 %) or BEHIND.
- **Infeasible goals get options, not a permanent "behind" (A-24).** The goal is marked infeasible when either:
  - it needs more than half of the trailing average income, or
  - the pace can never close the gap.

  It then suggests a realistic deadline: at the current pace when that is positive, otherwise at 30 % of the trailing average income. A realistic target is only suggested when the pace is positive — a flat or shrinking pace can't support one, so that case offers the deadline alone.
- **Goal cost at entry (A-23).** In quick add, a discretionary expense shows "~ 1.8 days of MacBook saving" under the amount.
  - The formula is `days = amount ÷ (requiredMonthly ÷ 30)`, in `BigDecimal`, HALF_UP, 1 decimal.
  - It is hidden for committed spending, income, transfers, when there is no goal, and when nothing is left to save.
  - It never blocks Save.
- **Money** is LKR minor units (`Long`) throughout. The divisions whose rounding the user sees — the required monthly saving, the projected and suggested dates, and the goal-cost hint — go through `BigDecimal` with an explicit rounding mode (ADR-0003). The pace average over complete months is plain integer division, which truncates by at most one minor unit.

Data flow



Reads come from Room; writes go to Firestore and come back through the sync listener (ADR-0004). `GetGoalProgressUseCase` also re-emits at local midnight, because months left, the pace window and the status all depend on "today".

The setup and detail screens keep calculations in their ViewModels. Setup validates with `Goal.validate()`, while detail maps `GoalProgress` and owns the what-if projection; Composables only render state and send events.

Edge cases handled

Case	Behaviour
First month of a goal	`NOT_ENOUGH_DATA`; the card shows the required saving and no verdict.
A month with no entries at all	Counts as a complete month of zero saving (ADR-0006). The pace stays honest, and the card stays neutral.
Deadline in the current month or already past	`monthsLeft` is at least 1, so the required saving is the whole remaining amount, never a division by zero.
Goal already reached	`remaining = 0`, so the required saving is 0 and the quick-add hint is hidden. The projected date is today when the pace is positive, and null otherwise. It is never infeasible, since that needs `remaining > 0`.
Pace ≤ 0 with something left to save	No projected date ("at this pace" has no answer). Marked infeasible, with a deadline suggested from 30 % of income and no target suggestion.
Archived goal read back after a sync	`GoalDto.isActive` maps the stored field explicitly. Before #157 the mapper read it as `active`, and archived goals came back as active.
Goal created offline	`startedAt` is client-set at creation, so progress has a start date even before the server `createdAt` syncs back (schema §6.5).
Two quick saves of a new goal	Serialised in `GoalRepositoryImpl`, so the "archive the previous active goal" decision isn't made twice.

Screenshots

TODO(M4) img/goal-dashboard-card.png — Dashboard goal card on seeded data	**TODO(M4) img/goal-quick-add-hint.png — Quick add showing the goal-cost hint**
Dashboard goal card: required vs actual monthly saving and the projected date (seeded data).	Quick add, Coffee & dining: the neutral goal-cost hint under the amount.

Tests

- `GetGoalProgressUseCaseTest`: the scenario figures and the S1–S3 acceptance figures (#162–#165).

- `GoalCostUseCaseTest`, `QuickAddViewModelTest`: the formula, rounding, and every visibility rule (#171).
- `DashboardViewModelTest`: the card states.
- `DtoMappingTest`, `RoomMappersTest`: deadline and start-date round-trips across time zones.